



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab experiment-10

A* (A-Star) Algorithm

Aim

To write a Python program to implement the A* search algorithm.

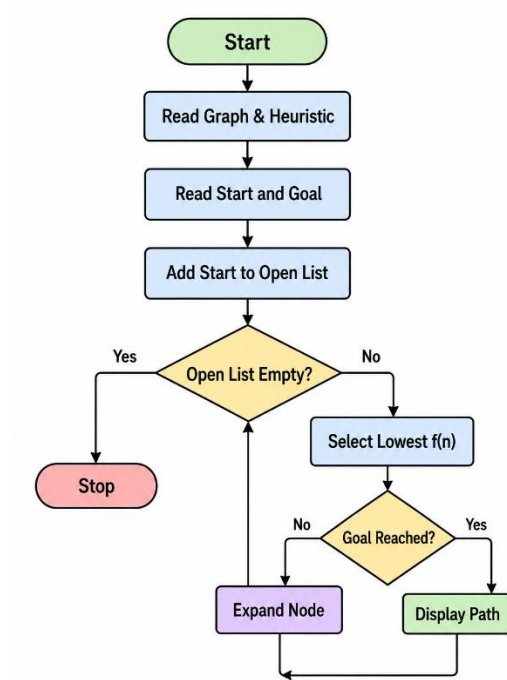
Objective

To find the shortest path between a start node and a goal node.

Algorithm

1. Start.
2. Read the graph and heuristic values.
3. Read the start and goal nodes.
4. Add the start node to the open list.
5. Select the node with the lowest $f(n) = g(n) + h(n)$.
6. Expand its neighbors and update their costs.
7. Repeat until the goal node is reached.
8. Display the shortest path.
9. Stop

Flowchart



Code

```
graph = {}
h = {}
n = int(input("Enter number of nodes: "))
for _ in range(n):
    node = input("Enter node: ")
    m = int(input(f"Enter number of neighbours of {node}: "))
    graph[node] = []
    for _ in range(m):
        nbr = input("Neighbour: ")
        cost = int(input("Cost: "))
        graph[node].append((nbr, cost))
    h[node] = int(input(f"Heuristic value of {node}: "))
start = input("Enter start node: ")
goal = input("Enter goal node: ")
```

```

open_list = [(h[start], 0, start, [start])]

visited = set()

while open_list:

    open_list.sort()

    f, g, node, path = open_list.pop(0)

    if node == goal:

        print("Shortest Path:", " -> ".join(path))

        print("Cost:", g)

        break

    if node in visited:

        continue

    visited.add(node)

    for nbr, cost in graph[node]:

        if nbr not in visited:

            open_list.append((g + cost + h[nbr], g + cost, nbr, path + [nbr]))

```

Output

```

1 graph = {}
2 h = {}
3
4 n = int(input("Enter number of nodes: "))
5
6 for _ in range(n):
7     node = input("Enter node: ")
8     m = int(input(f"Enter number of neighbours of {node}: "))
9     graph[node] = []
10
11     for _ in range(m):
12         nbr = input("Neighbour: ")
13         cost = int(input("Cost: "))
14         graph[node].append((nbr, cost))
15
16 h[node] = int(input(f"Heuristic value of {node}: "))

```

Python Debug Console

```

Cost: 5
Heuristic value of B: 6
Enter node: C
Enter number of neighbours of C: 1
Neighbour: D
Cost: 1
Heuristic value of C: 2
Enter node: D
Enter number of neighbours of D: 0
Heuristic value of D: 0
Enter start node: A
Enter goal node: D
Shortest Path: A -> C -> D
Cost: 5

```

Tip: Use [create-agent](#) to scaffold a custom agent for your workflow.

+ exp 10

Describe what to bu...

+ ... Default ...

Result

The shortest path from the start node to the goal node was found successfully using the A* algorithm.

Conclusion

The A* algorithm successfully found the optimal path by using both the path cost and heuristic values.